

Computer Networks (EC-330)

Lecture 8 – Network Security
Muhammad Umar Farooq, Ph.D.
ufarooq@ceme.nust.edu.pk

Network Security

Chapter goals:

- ❖ understand principles of network security:
 - cryptography and its *many* uses beyond “confidentiality”
 - authentication
 - message integrity
- ❖ security in practice:
 - firewalls and intrusion detection systems
 - security in application, transport, network, link layers

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity, authentication

8.4 Securing e-mail

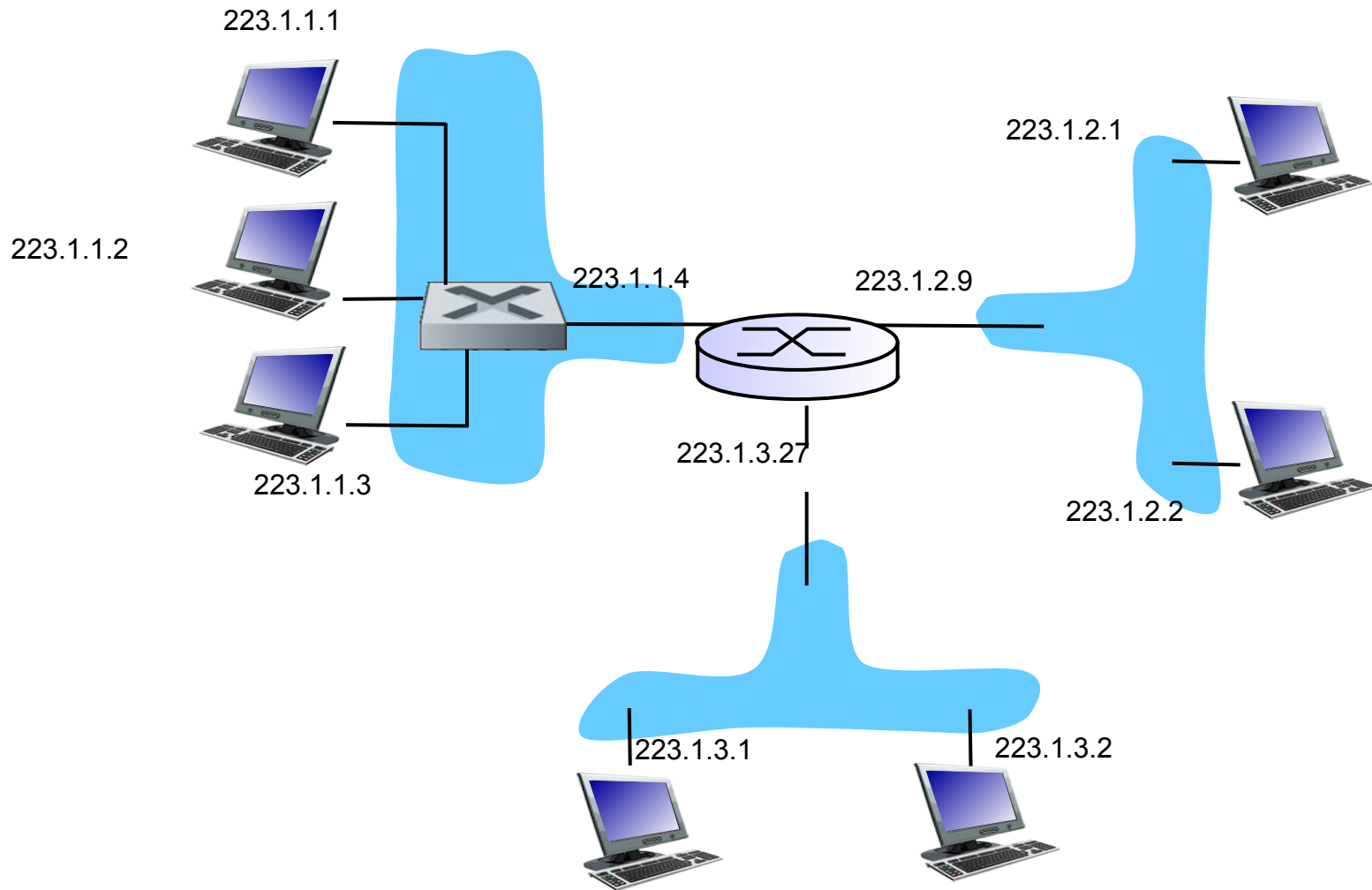
8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

Example Scenarios



What is network security?

confidentiality: only sender, intended receiver should “understand” message contents

- sender encrypts message
- receiver decrypts message

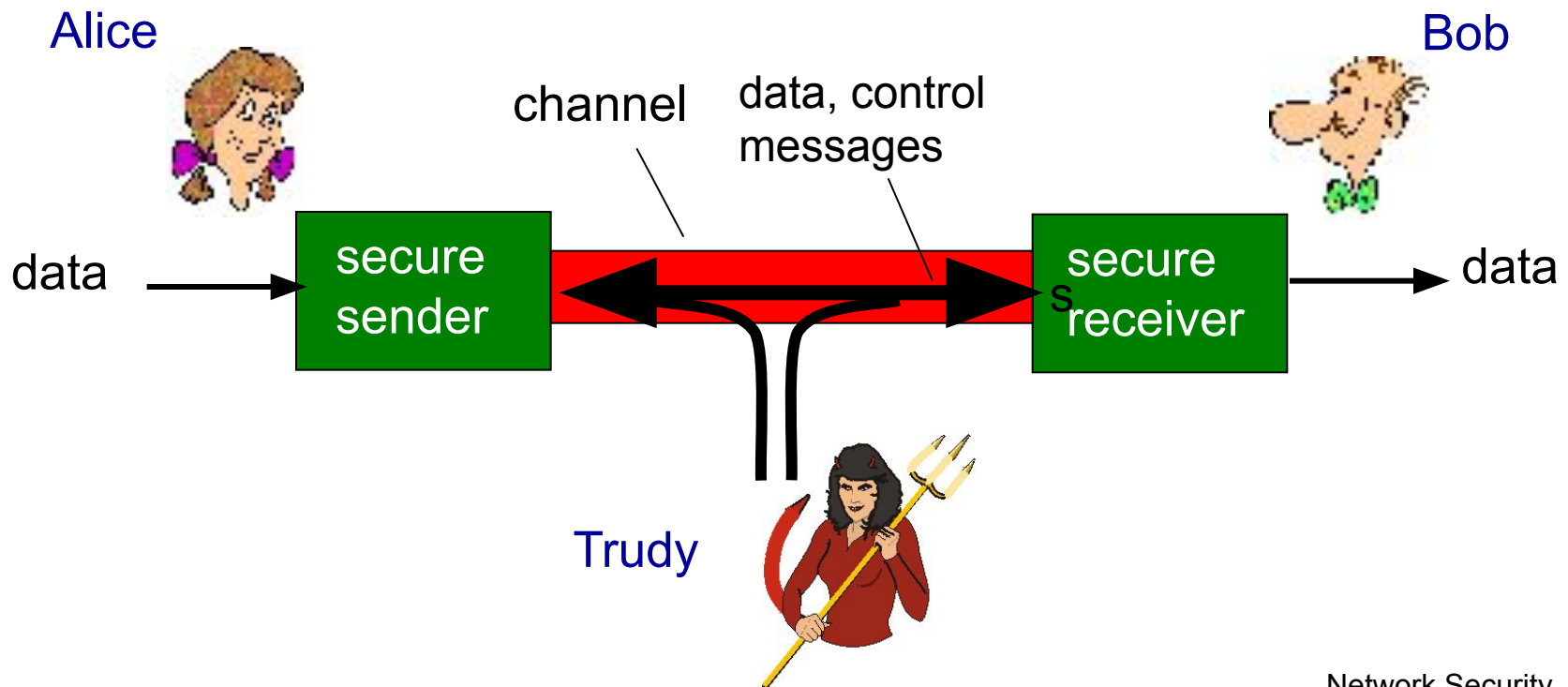
authentication: sender, receiver want to confirm identity of each other

message integrity: sender, receiver want to ensure message not altered (in transit, or afterwards) without detection

access and availability: services must be accessible and available to users

Friends and enemies: Alice, Bob, Trudy

- ❖ well-known in network security world
- ❖ Bob, Alice want to communicate “securely”
- ❖ Trudy (intruder) may intercept, delete, add messages



Who might Bob, Alice be?

- ❖ Web browser/server for electronic transactions (e.g., on-line purchases)
- ❖ on-line banking client/server
- ❖ DNS servers
- ❖ routers exchanging routing table updates
- ❖ other examples?

There are bad guys out there!

Q: What can a “bad guy” do?

A: A lot! See section 1.6

- *eavesdrop*: intercept messages
- actively *insert* messages into connection
- *impersonation*: can fake (spoof) source address in packet (or any field in packet)
- *hijacking*: “take over” ongoing connection by removing sender or receiver, inserting himself in place
- *denial of service*: prevent service from being used by others (e.g., by overloading resources)

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity, authentication

8.4 Securing e-mail

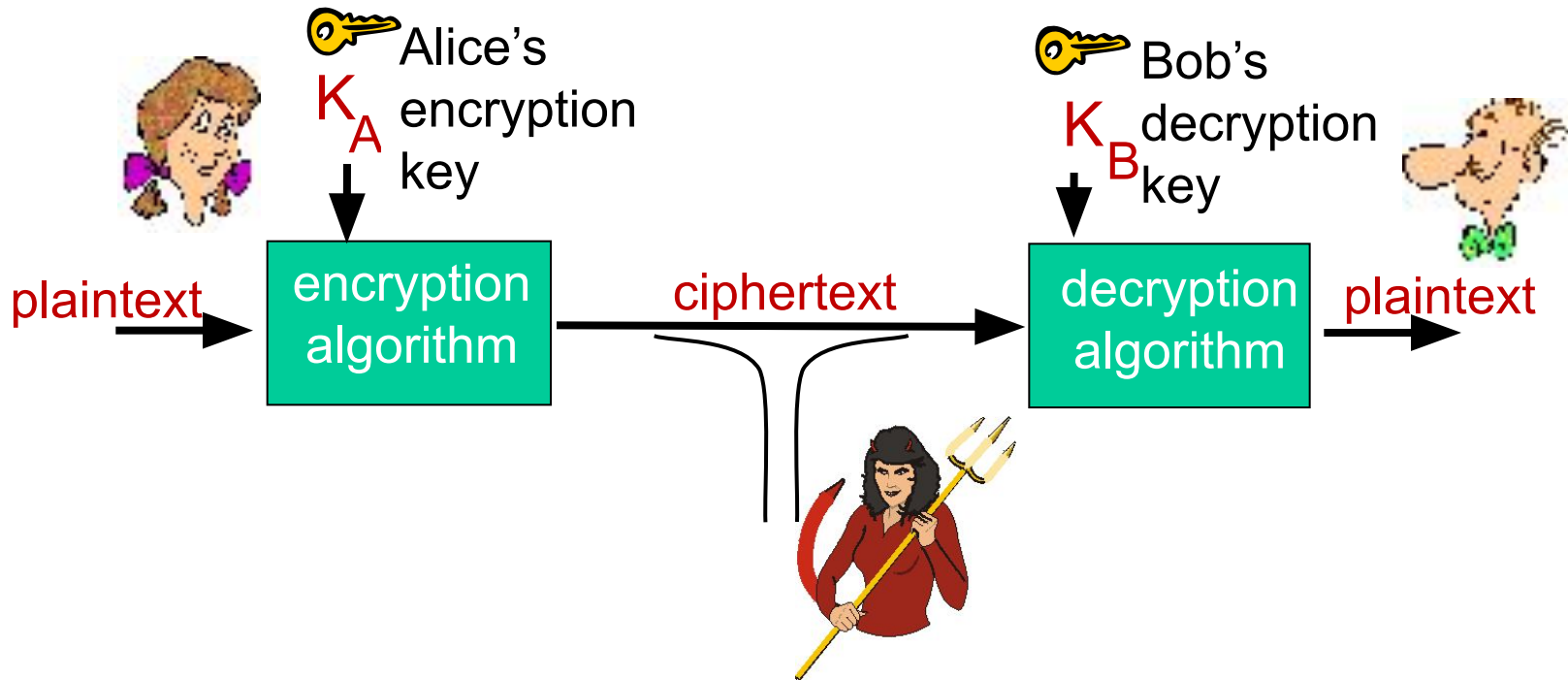
8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

The language of cryptography

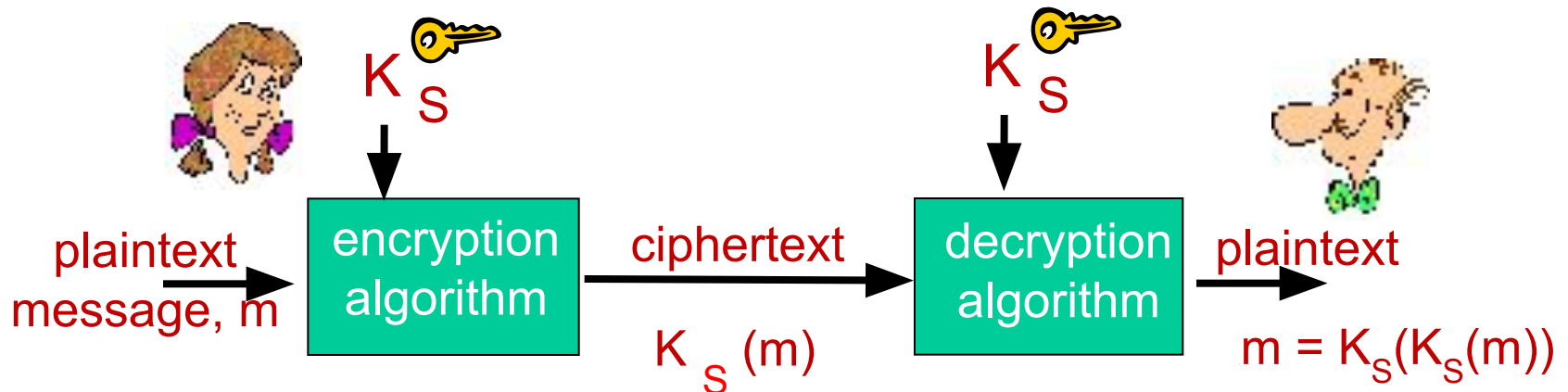


m plaintext message

$K_A(m)$ ciphertext, encrypted with key K_A

$m = K_B(K_A(m))$

Symmetric key cryptography



symmetric key crypto: Bob and Alice share same (symmetric) key: K_S

- ❖ e.g., key is knowing substitution pattern in mono alphabetic substitution cipher

Q: how do Bob and Alice agree on key value?

Simple encryption schemes

Caesar Cipher (earliest technique): work by taking each letter in the plaintext and substituting the letter that is k letters later (allowing wraparound; that is, having the letter z followed by the letter a) in the alphabet.

e.g.:

GREETINGS BOB, ALICE
↓
JUHHLWLQJ V ERE, DOLFH

 *Encryption key:* $K = 3$

Simple encryption scheme

substitution cipher: substituting one thing for another

- *monoalphabetic cipher*: substitute one letter for another

plaintext:
abcdefghijklmnopqrstuvwxyz
↓ ↓
ciphertext:

mnbvcxz asdfghjklpoiuytrewq

e.g.: Plaintext: GREETINGS BOB, ALICE
ciphertext: ZOCCUSJZI NKN, MGSBC

🔑 *Encryption key*: mapping from set of 26 letters

to set of 26 letters

Breaking an encryption scheme

❖ cipher-text only attack:

Trudy has ciphertext she can analyze

▪ two approaches:

- brute force: search through all keys
- statistical analysis
 - E, T, IN, IT, THE etc.

❖ known-plaintext attack:

Trudy has plaintext corresponding to ciphertext

- e.g., in monoalphabetic cipher, Trudy determines pairings for a,l,i,c,e,b,o,

❖ chosen-plaintext attack:

Trudy can get ciphertext for chosen plaintext

A more sophisticated encryption approach

❖ Polyalphabet Cipher:

- n substitution ciphers, $M_1, M_2, \dots, M_n$
- ❖ cycling pattern:
 - e.g., $n=4$: M_1, M_3, M_4, M_3, M_2 ; M_1, M_3, M_4, M_3, M_2 ; ..
- ❖ for each new plaintext symbol, use subsequent substitution pattern in cyclic pattern
 - dog: d from M_1 , o from M_3 , g from M_4



Encryption key: n substitution ciphers, and cyclic pattern

- key need not be just n-bit pattern

Block Ciphers

- ❖ Current symmetric key encryption schemes
 - **Block Ciphers**, Stream Ciphers
- ❖ Block Ciphers: Message to be encrypted is processed in blocks of k bits
 - For $K=64$, message broken into 64 bit blocks, and encrypted independently
 - One-to-one mapping to map the k -bit block of cleartext to a k -bit block of ciphertext

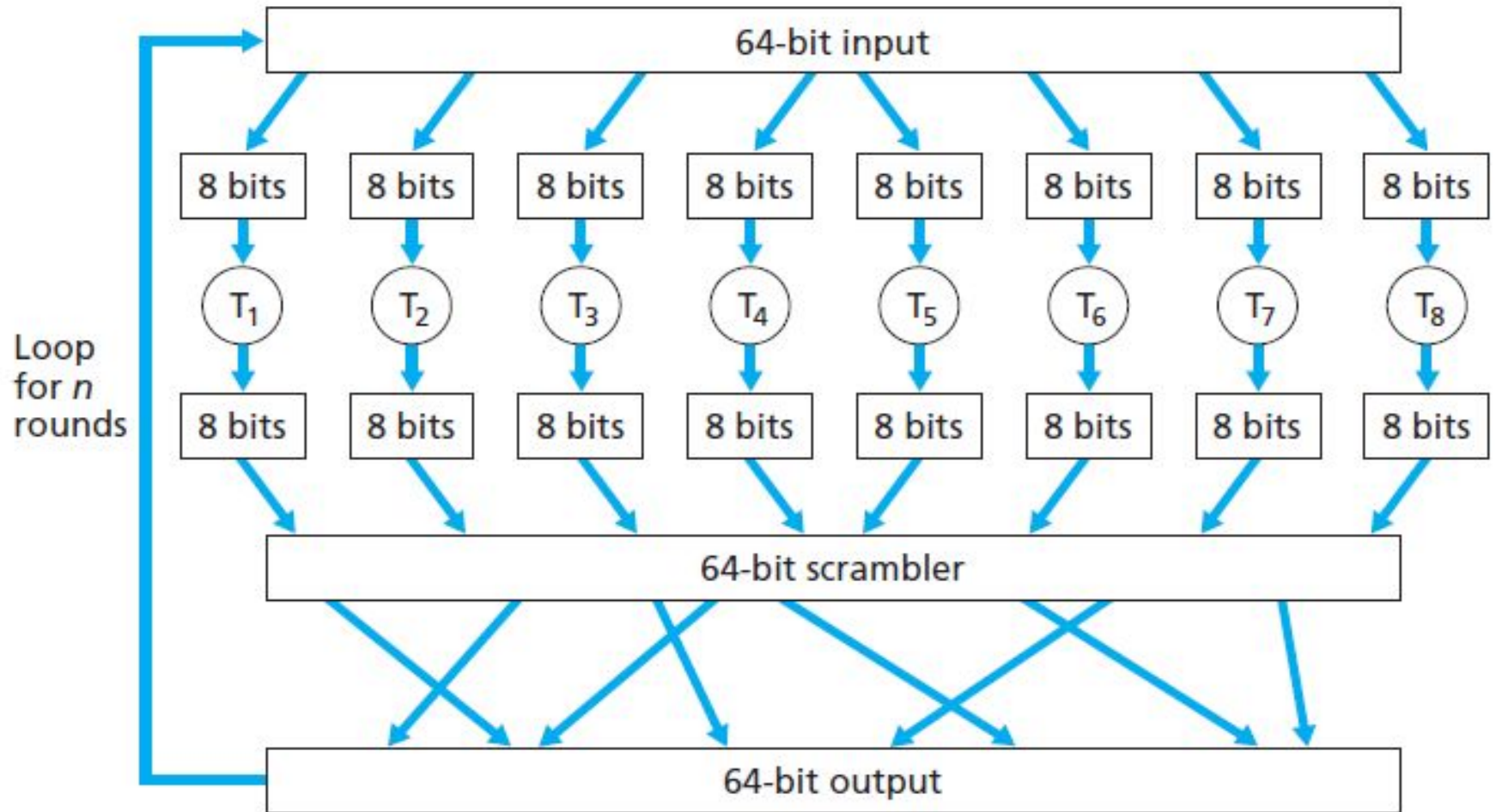
Input	Output	Input	Output
000	110	100	011
001	111	101	010
010	101	110	000
011	100	111	001

Message: 010110001111
Encrypted: 101000111001

Block Ciphers

- ❖ block ciphers typically use functions that simulate randomly permuted
- ❖ The function first breaks a 64-bit block into 8 chunks,
- ❖ with each chunk consisting of 8 bits
- ❖ Each 8-bit chunk is processed by an 8-bit to 8-bit table, which is of manageable size
- ❖ Next, the 8 output chunks are reassembled into a 64-bit block (with positions scrambled)
- ❖ This output is fed back to the 64-bit input, where another cycle begins.
- ❖ After n such cycles, the function provides a 64-bit block of ciphertext.
- ❖ Key = Eight permutation tables

Block Ciphers



Cipher Block Chaining

- ❖ For **identical blocks**, a block cipher would produce the same ciphertext
 - An attacker could potentially guess the cleartext when it sees identical ciphertext blocks
- ❖ mix some randomness into the ciphertext so that identical plaintext blocks produce different ciphertext blocks
 - The sender creates a random k -bit number $r(i)$ for the i th block and calculates
$$c(i) = K_S(m(i) \oplus r(i))$$
- ❖ **Key:** $c(i)$ and $r(i)$

Cipher Block Chaining

Message: 010110001111
 Encrypted: 101000111001

Input	Output	Input	Output
000	110	100	011
001	111	101	010
010	101	110	000
011	100	111	001

Message (m): 010110001111
 IV (r): 100110101001
 $m \oplus r$: 110000100110
 $C(m \oplus r)$: 000110011000

$C(m \oplus r)$: 000110011000
 $m \oplus r$: 110000100110
 IV (r): 100110101001
 Message (m): 010110001111

Message (m): 010110001111
 IV (r): 000110011000
 $m \oplus r$: 010000010111
 $C(m \oplus r)$: 101110101001

$C(m \oplus r)$: 101110101001
 $m \oplus r$: 010000010111
 IV (r): 000110011000
 Message (m): 010110001111

Symmetric key crypto: DES (Self Study)

DES: Data Encryption Standard

- ❖ US encryption standard [NIST 1993]
- ❖ 56-bit symmetric key, 64-bit plaintext input
- ❖ block cipher with cipher block chaining
- ❖ how secure is DES?
 - DES Challenge: 56-bit-key-encrypted phrase decrypted (brute force) in less than a day
 - no known good analytic attack
- ❖ making DES more secure:
 - 3DES: encrypt 3 times with 3 different keys

AES: Advanced Encryption Standard (Self Study)

- ❖ symmetric-key NIST standard, replaced DES (Nov 2001)
- ❖ processes data in 128 bit blocks
- ❖ 128, 192, or 256 bit keys
- ❖ brute force decryption (try each key) taking 1 sec on DES, takes 149 trillion years for AES

Public Key Cryptography



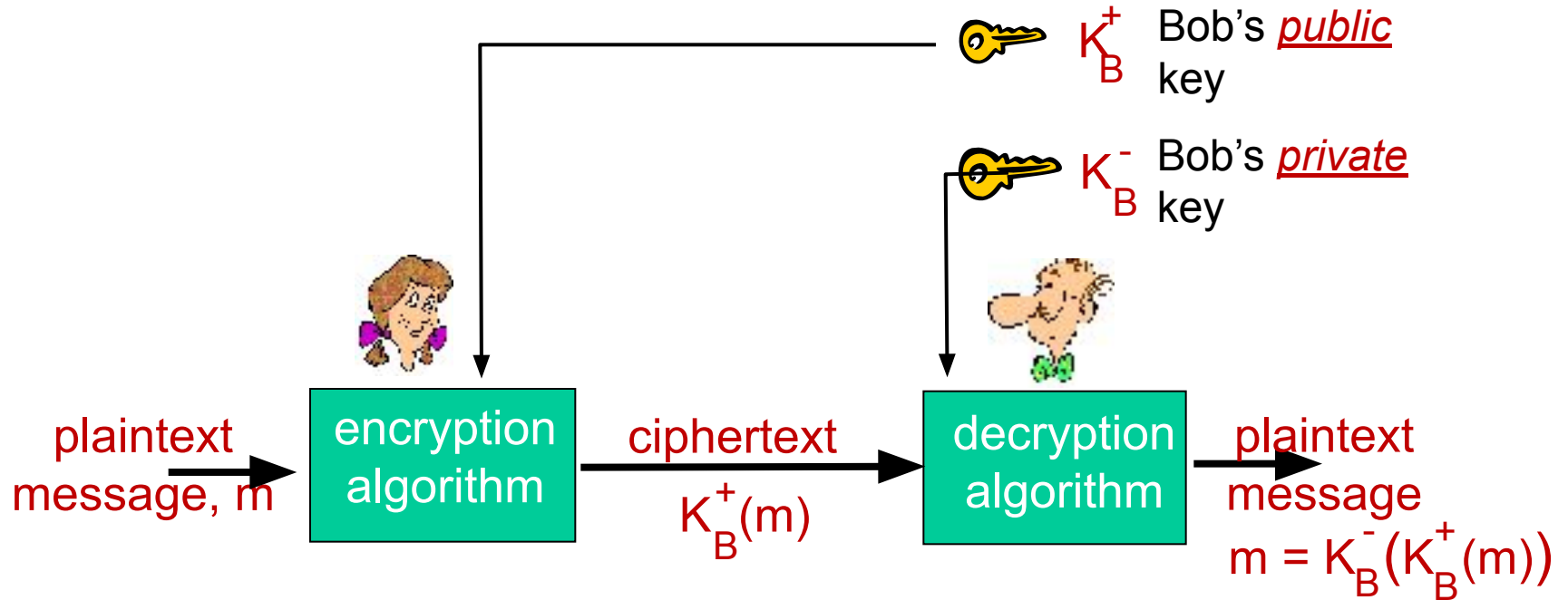
symmetric key crypto

- ❖ requires sender, receiver know shared secret key
- ❖ Q: how to agree on key in first place (particularly if never “met”)?

public key crypto

- ❖ radically different approach [Diffie-Hellman76, RSA78]
- ❖ sender, receiver do *not* share secret key
- ❖ *public* encryption key known to *all*
- ❖ *private* decryption key known only to receiver

Public key cryptography



Public key encryption algorithms

requirements:

① need $K_B^+(\cdot)$ and $K_B^-(\cdot)$ such that

$$K_B^-(K_B^+(m)) = m$$

② given public key K_B^+ , it should be impossible to compute private key K_B^-

RSA: Rivest, Shamir, Adelson
algorithm

RSA: getting ready

- ❖ message: just a bit pattern
- ❖ bit pattern can be uniquely represented by an integer number
- ❖ thus, encrypting a message is equivalent to encrypting a number.

example:

- ❖ $m = 10010001$. This message is uniquely represented by the decimal number 145.
- ❖ to encrypt m , we encrypt the corresponding number, which gives a new number (the ciphertext).

RSA: Creating public/private key pair

1. choose two large prime numbers p, q .
(e.g., 1024 bits each)
2. compute $n = pq$, $z = (p-1)(q-1)$
3. choose e (with $e < n$) that has no common factors with z (e, z are “relatively prime”).
4. choose d such that $ed - 1$ is exactly divisible by z .
(in other words: $ed \bmod z = 1$).
5. public key is (n, e) . private key is (n, d) .


RSA: encryption, decryption

0. given (n, e) and (n, d) as computed above

1. to encrypt message $m (< n)$,
compute $c = m^e \bmod n$

2. to decrypt received bit pattern, c , compute

$$m = c^d \bmod n$$

magic happens!

$$m = \underbrace{(m^e \bmod n)}_c^d \bmod n$$

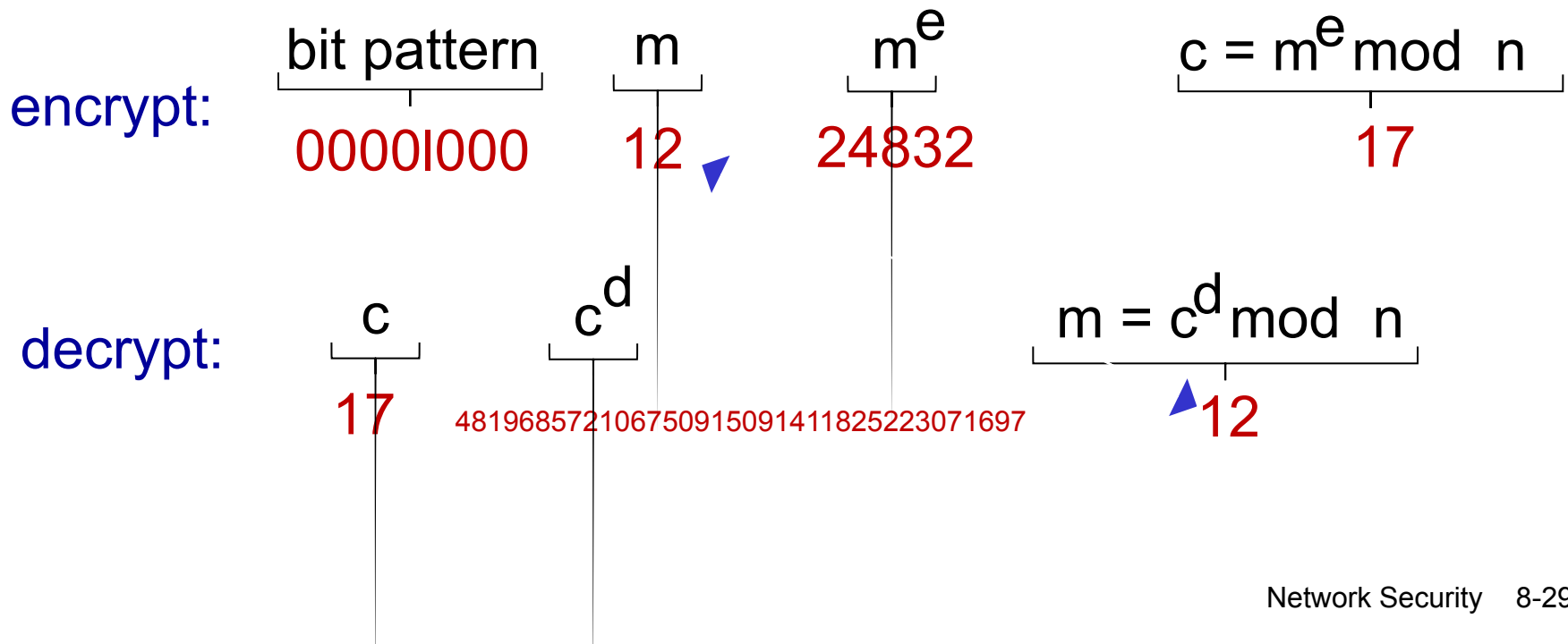
RSA example:

Bob chooses $p=5$, $q=7$. Then $n=35$, $z=24$.

$e=5$ (so e , z relatively prime).

$d=29$ (so $ed-1$ exactly divisible by z).

encrypting 8-bit messages.



RSA: another important property

The following property will be *very* useful later:

$$\underbrace{K_B^-(K_B^+(m))}_{\substack{\text{use public key} \\ \text{first, followed by} \\ \text{private key}}} = m = \underbrace{K_B^+(K_B^-(m))}_{\substack{\text{use private key} \\ \text{first, followed by} \\ \text{public key}}}$$

result is the same!

Why is RSA secure?

- ❖ suppose you know Bob's public key (n,e) . How hard is it to determine d ?
- ❖ essentially need to find factors of n without knowing the two factors p and q
 - fact: factoring a big number is hard

RSA in practice: session keys

- ❖ exponentiation in RSA is computationally intensive
- ❖ DES is at least 100 times faster than RSA
- ❖ use public key crypto to establish secure connection, then establish second key – symmetric session key – for encrypting data

session key, K_s

- ❖ Bob and Alice use RSA to exchange a symmetric key K_s
- ❖ once both have K_s , they use symmetric key cryptography

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity, *authentication*

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”



Failure scenario??



Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”

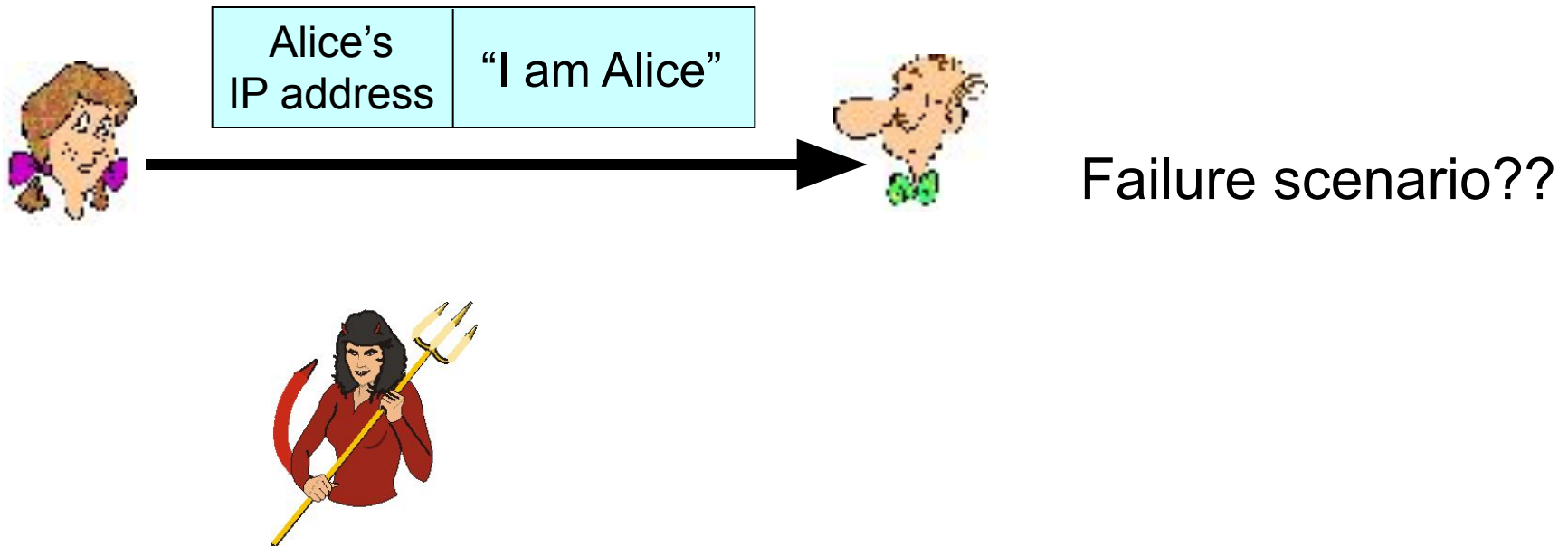


“I am Alice”

in a network,
Bob can not “see” Alice,
so Trudy simply declares
herself to be Alice

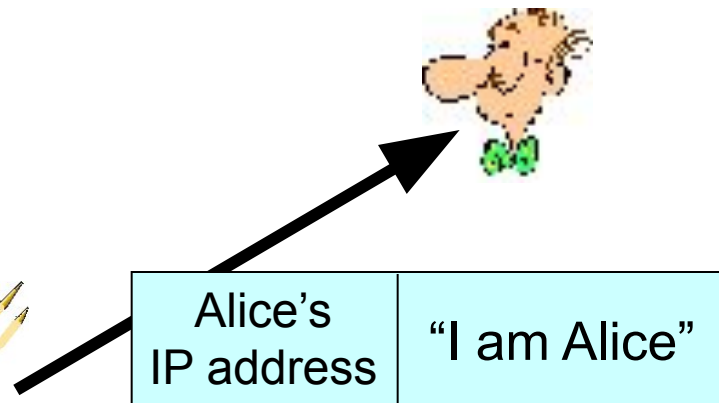
Authentication: another try

Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address



Authentication: another try

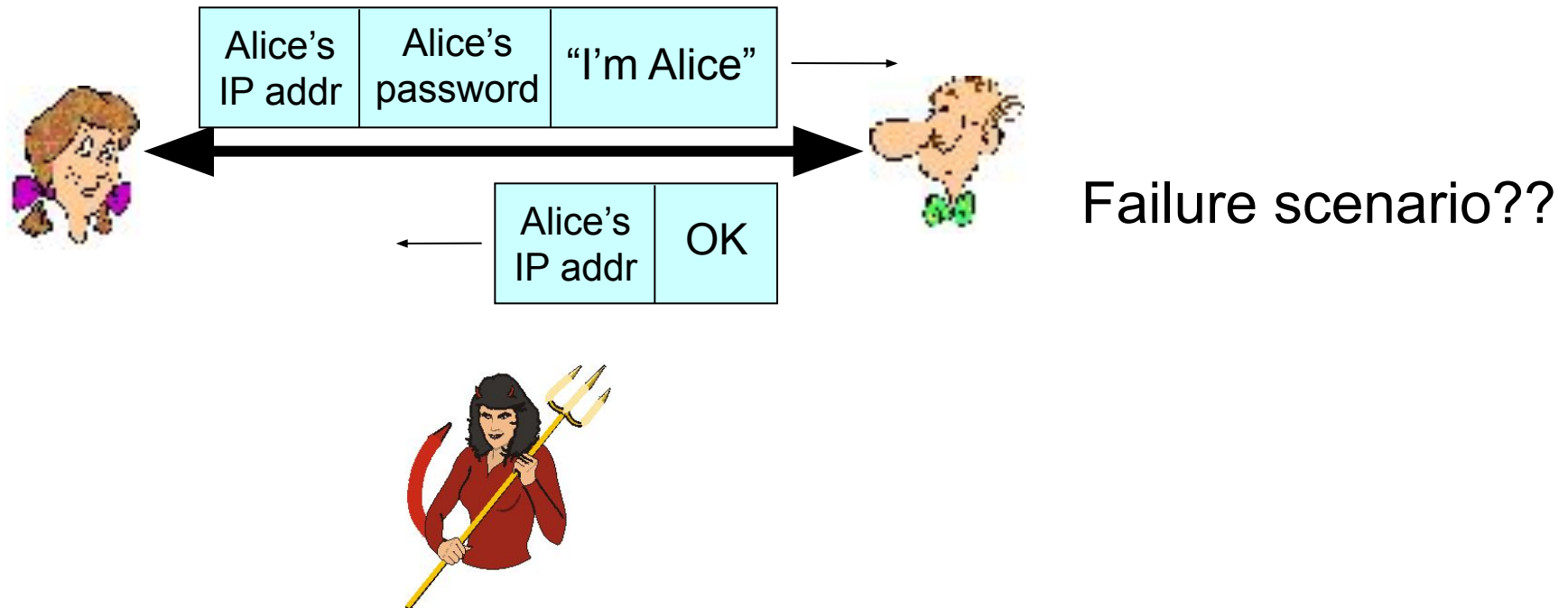
Protocol ap2.0: Alice says “I am Alice” in an IP packet containing her source IP address



Trudy can create a packet “spoofing” Alice’s address

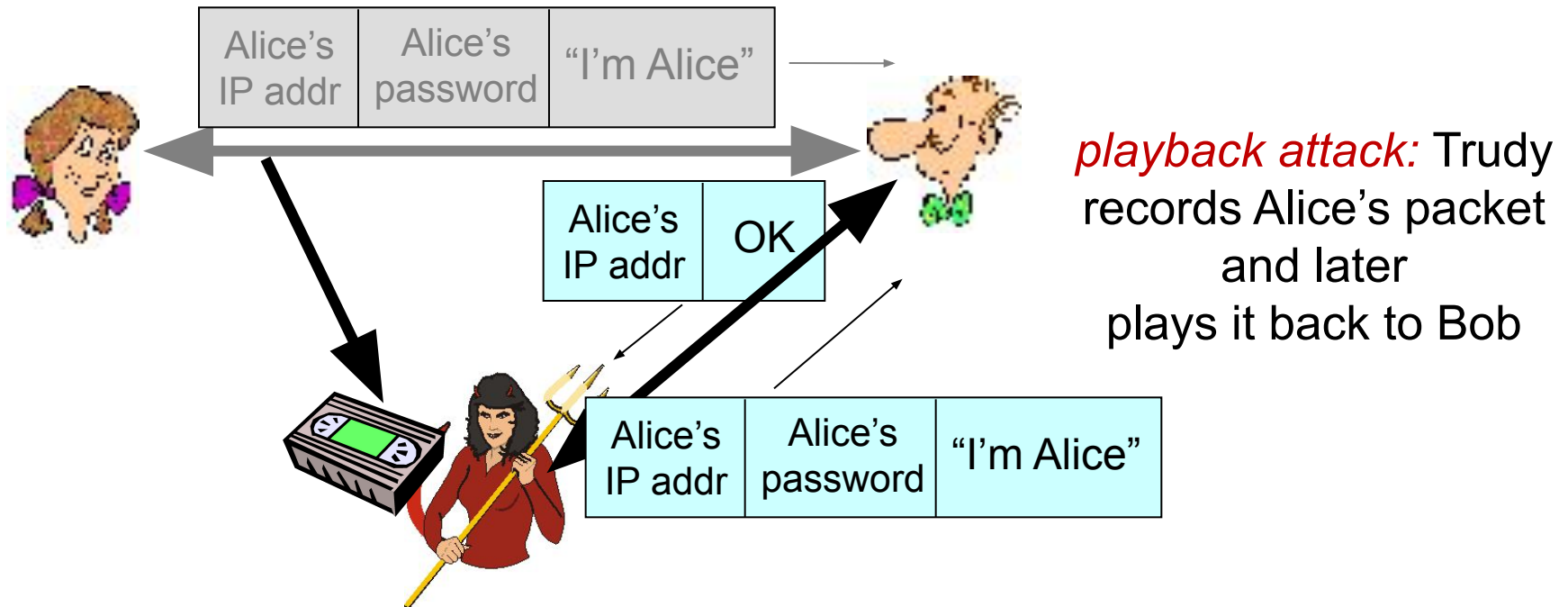
Authentication: another try

Protocol ap3.0: Alice says “I am Alice” and sends her secret password to “prove” it.



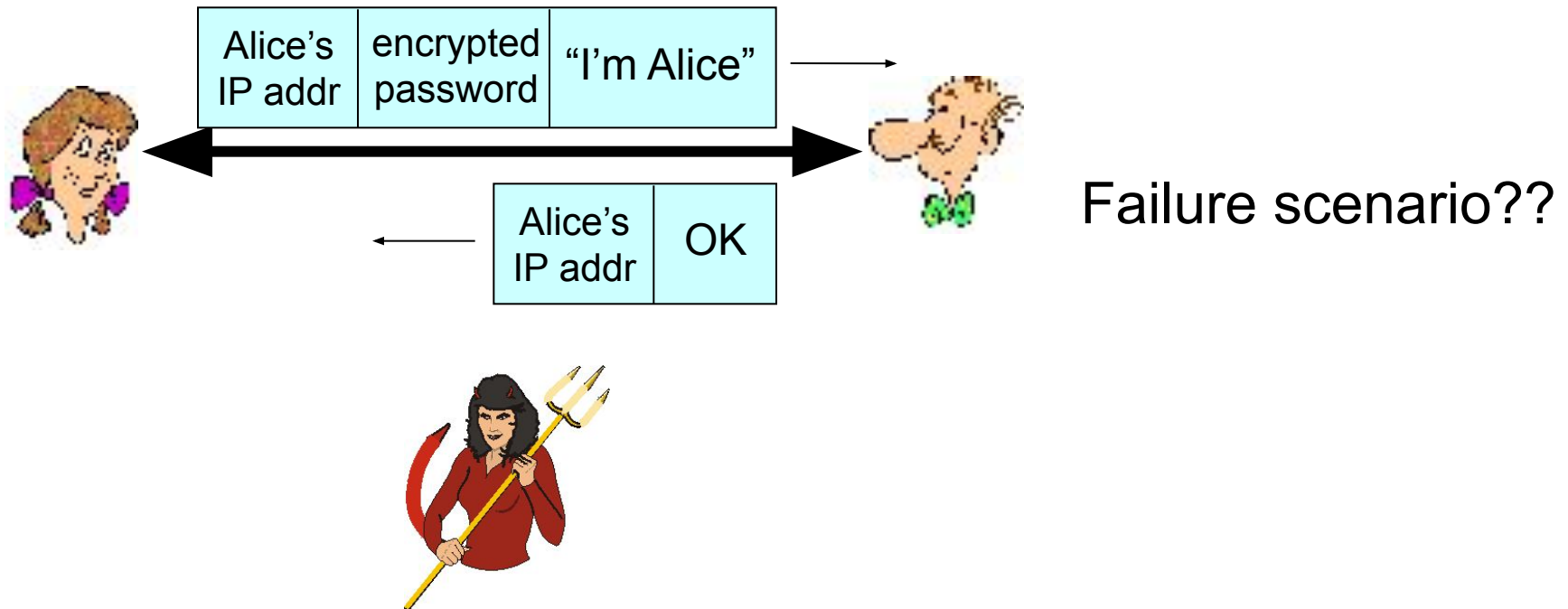
Authentication: another try

Protocol ap3.0: Alice says “I am Alice” and sends her secret password to “prove” it.



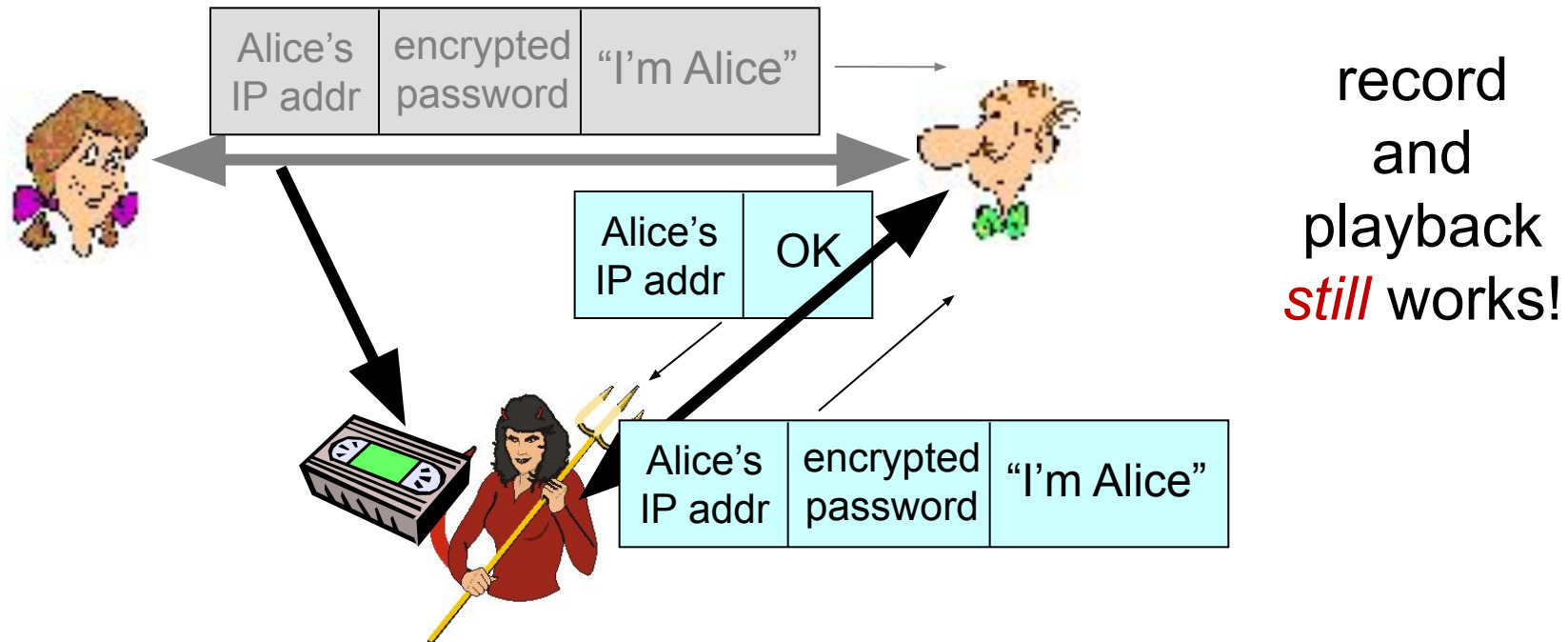
Authentication: yet another try

Protocol ap3.1: Alice says “I am Alice” and sends her *encrypted* secret password to “prove” it.



Authentication: yet another try

Protocol ap3.1: Alice says “I am Alice” and sends her *encrypted* secret password to “prove” it.

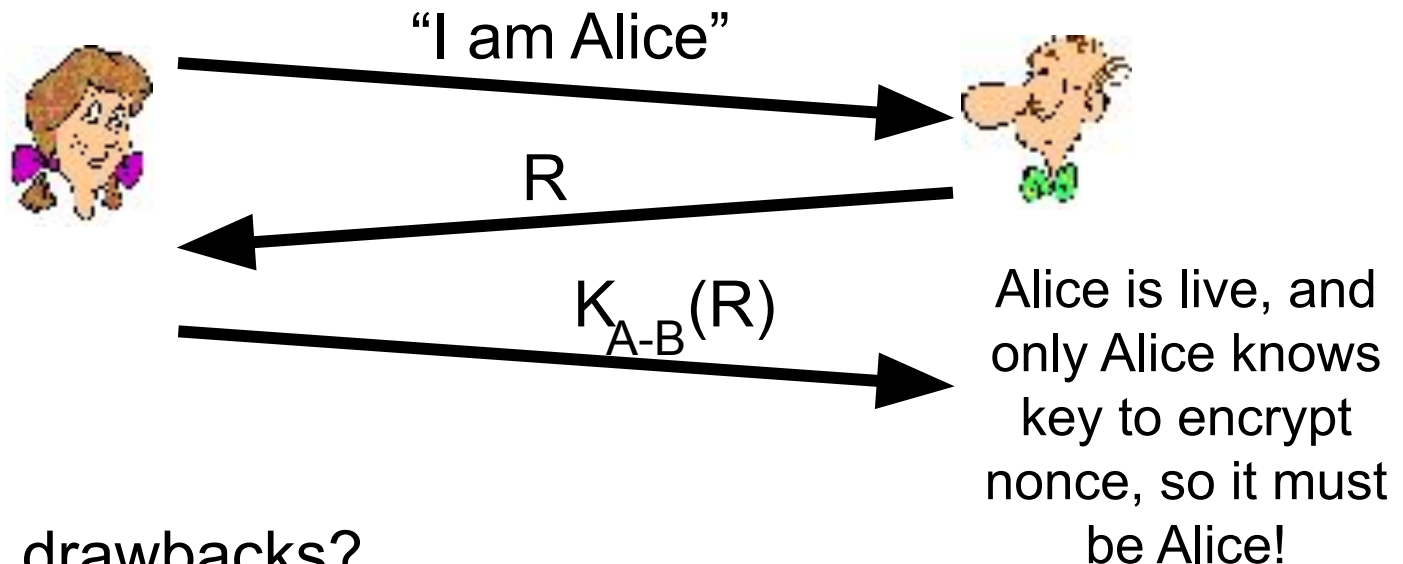


Authentication: yet another try

Goal: avoid playback

nonce ^{attack} number (R) used only

ap4.0: to prove Alice “live”, Bob sends Alice **nonce**, R. Alice must return R, encrypted with shared secret key



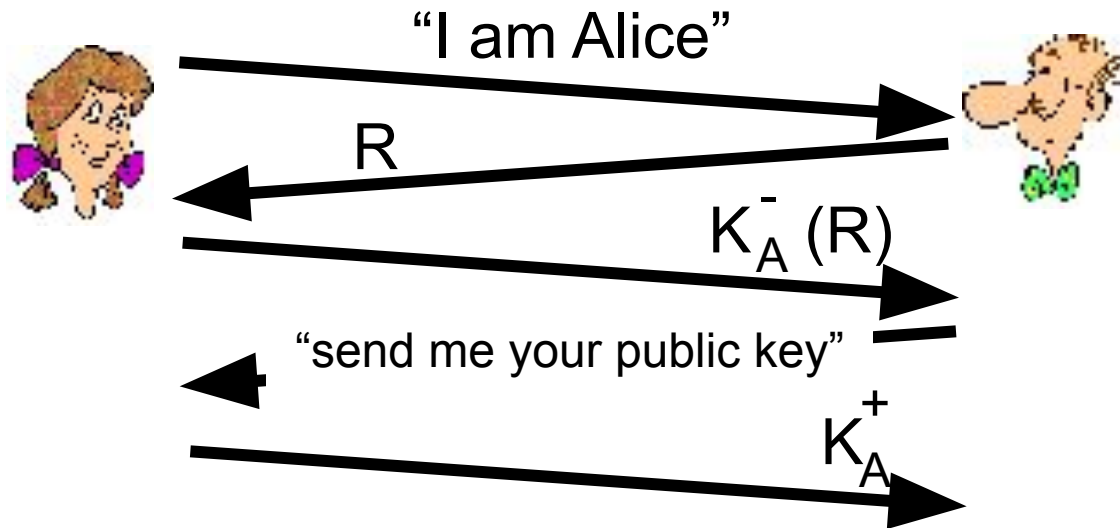
Failures, drawbacks?

Authentication: ap5.0

ap4.0 requires shared symmetric key

❖ can we authenticate using public key techniques?

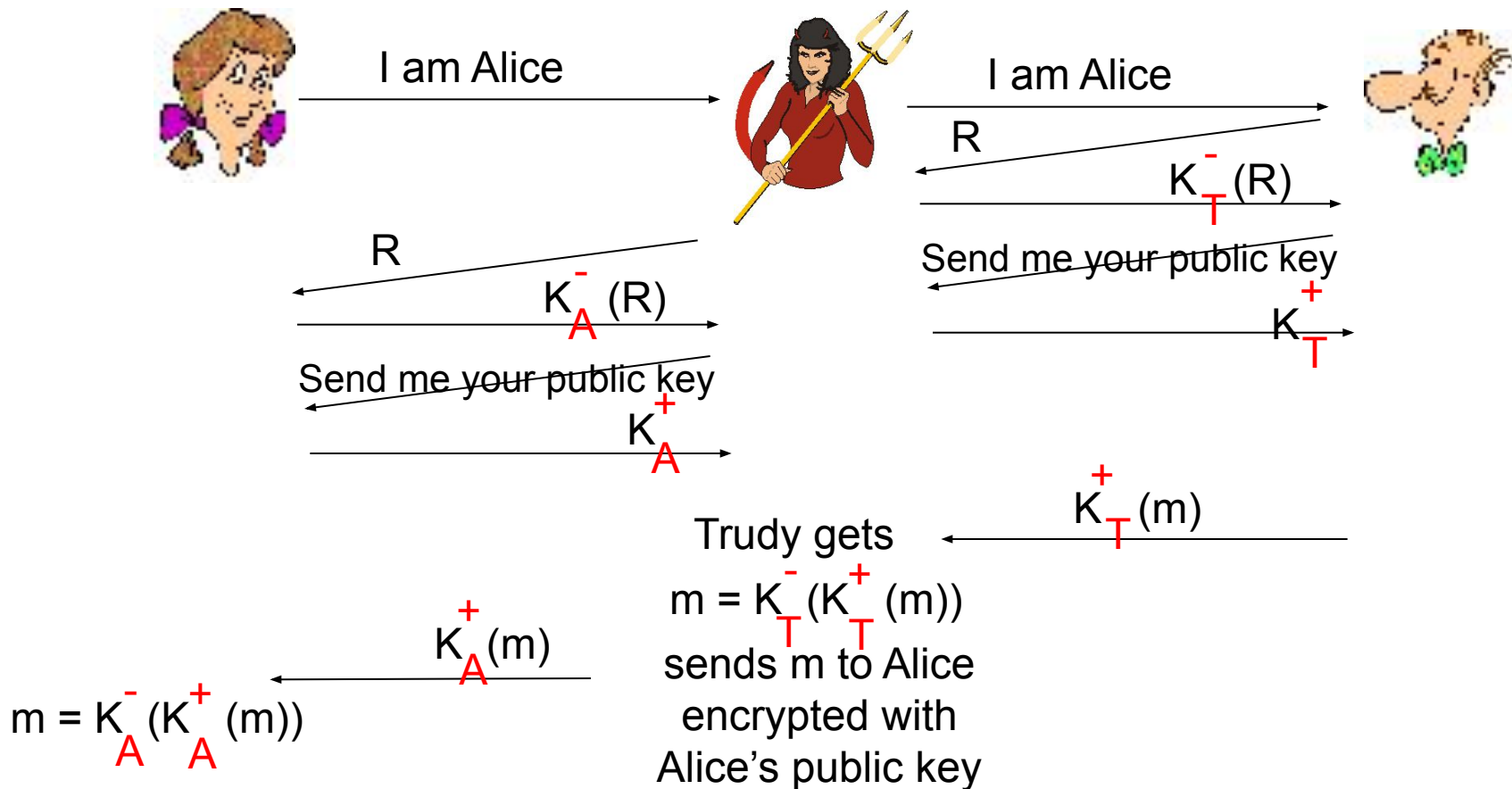
ap5.0: use nonce, public key cryptography



Bob computes
 $K_A^+(K_A^-(R)) = R$
and knows only Alice
could have the private
key, that encrypted R
such that
 $K_A^+(K_A^-(R)) = R$

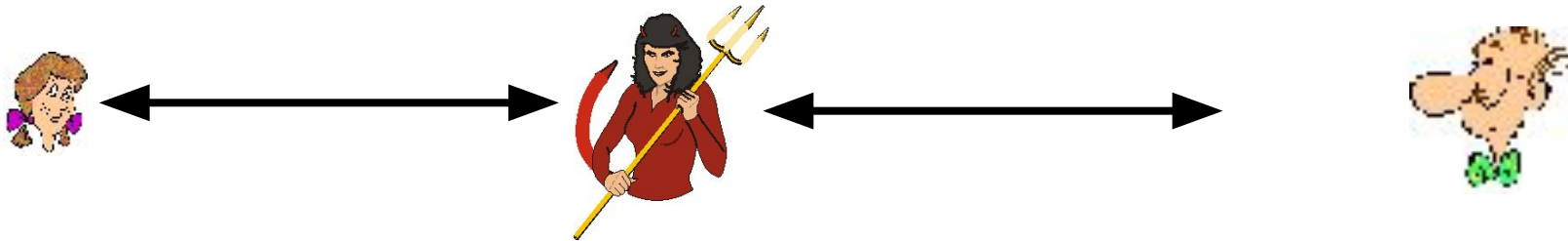
ap5.0: security hole

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)



ap5.0: security hole

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)



difficult to detect:

- ❖ Bob receives everything that Alice sends, and vice versa. (e.g., so Bob, Alice can meet one week later and recall conversation!)
- ❖ problem is that Trudy receives all messages as well!

Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 *Message integrity*, authentication

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

Digital signatures

cryptographic technique analogous to hand-written signatures:

- ❖ sender (Bob) digitally signs document, establishing he is document owner/creator.
- ❖ *verifiable, nonforgeable*: recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed document

Digital signatures

simple digital signature for message m :

- ❖ Bob signs m by encrypting with his private key K_B^- , creating “signed” message, $K_B^-(m)$

Bob's message, m

Dear Alice
Oh, how I have missed
you. I think of you all the
time! ... (blah blah blah)
Bob

 K_B^- Bob's private
key

Public key
encryption
algorithm

$m, K_B^-(m)$

Bob's message, m ,
signed (encrypted)
with his private key

Digital signatures

- ❖ suppose Alice receives msg m , with signature: $m, K_B^-(m)$
- ❖ Alice verifies m signed by Bob by applying Bob's public key K_B to $K_B^-(m)$ then checks $K_B^+(K_B^-(m)) = m$.
- ❖ If $K_B^+(K_B^-(m)) = m$, whoever signed m must have used Bob's private key.

Alice thus verifies that:

- ü Bob signed m
- ü no one else signed m
- ü Bob signed m and not m'

non-repudiation:

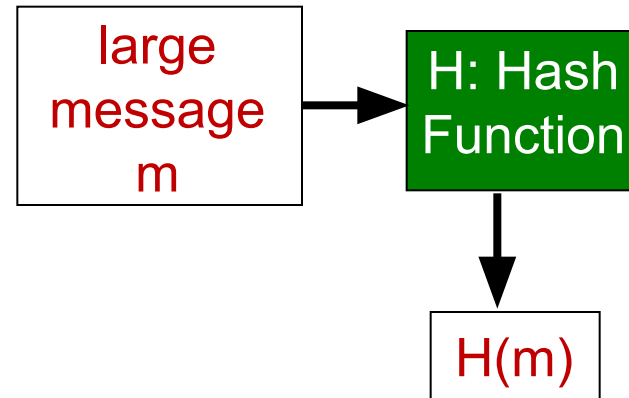
- ü Alice can take m , and signature $K_B^-(m)$ to court and prove that Bob signed m

Message digests

computationally expensive to
public-key-encrypt long
messages

goal: fixed-length, easy-
to-compute digital
“fingerprint”

- ❖ apply **hash function** H to m ,
get fixed size message
digest, $H(m)$.



Hash function properties:

- ❖ many-to-1
- ❖ produces fixed-size msg
digest (fingerprint)
- ❖ given message digest x ,
computationally infeasible to
find m such that $x = H(m)$

Internet checksum: poor crypto hash function

Internet checksum has some properties of hash function:

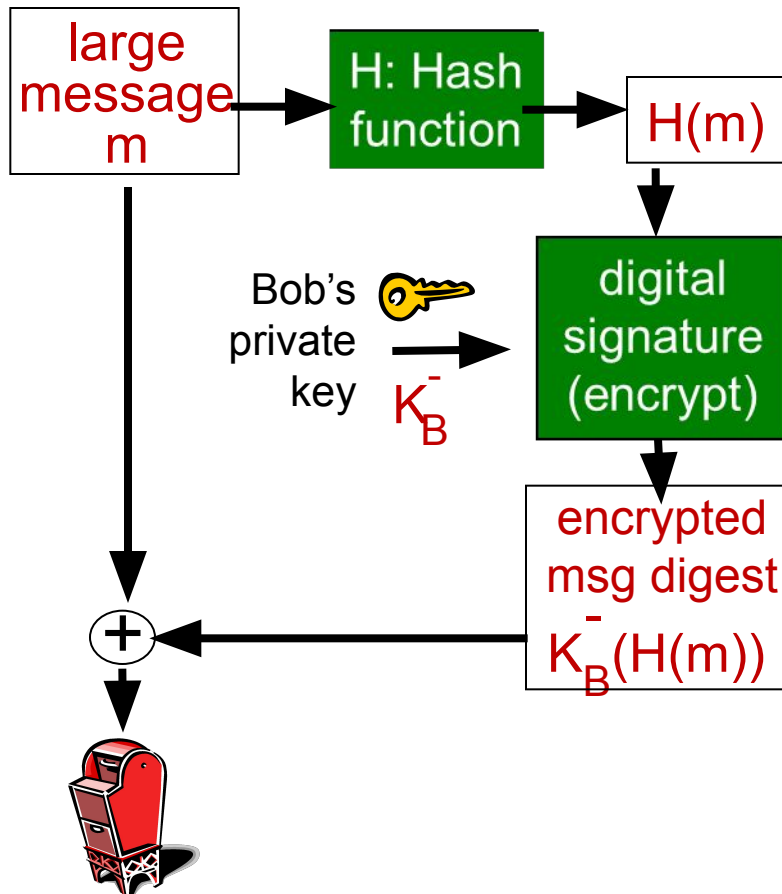
- produces fixed length digest (16-bit sum) of message
- is many-to-one

But given message with given hash value, it is easy to find another message with same hash value:

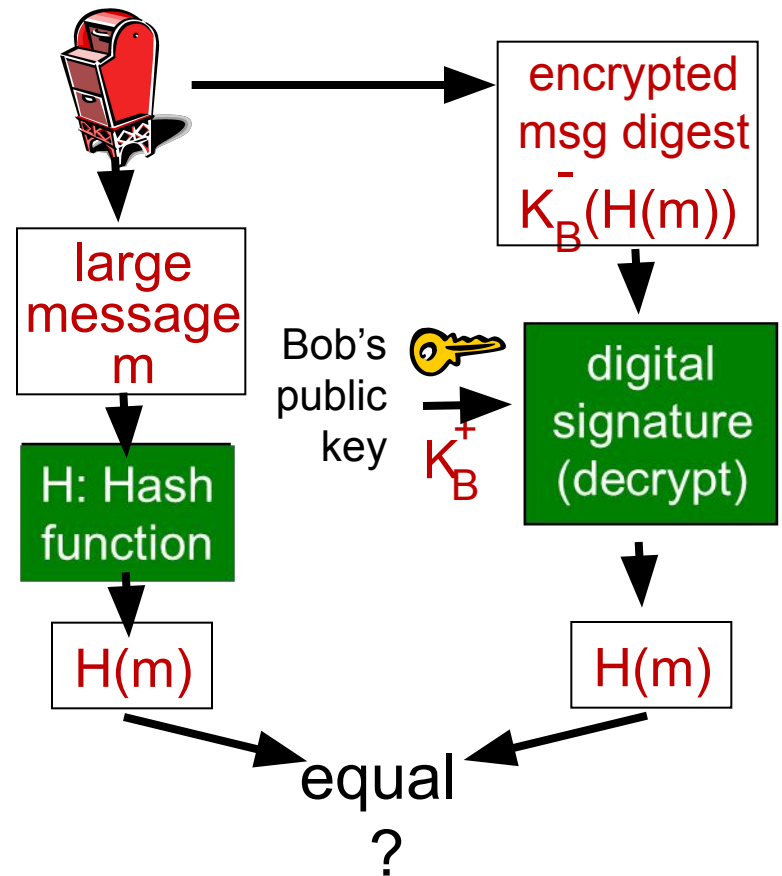
<u>message</u>	<u>ASCII format</u>		<u>message</u>	<u>ASCII format</u>
I O U 1	49 4F 55 31		I O U <u>9</u>	49 4F 55 <u>39</u>
0 0 . 9	30 30 2E 39		0 0 . <u>1</u>	30 30 2E <u>31</u>
9 B O B	39 42 D2 42		9 B O B	39 42 D2 42
B2 C1 D2 AC		different messages but identical checksums!	B2 C1 D2 AC	

Digital signature = signed message digest

Bob sends digitally signed message:



Alice verifies signature, integrity of digitally signed message:

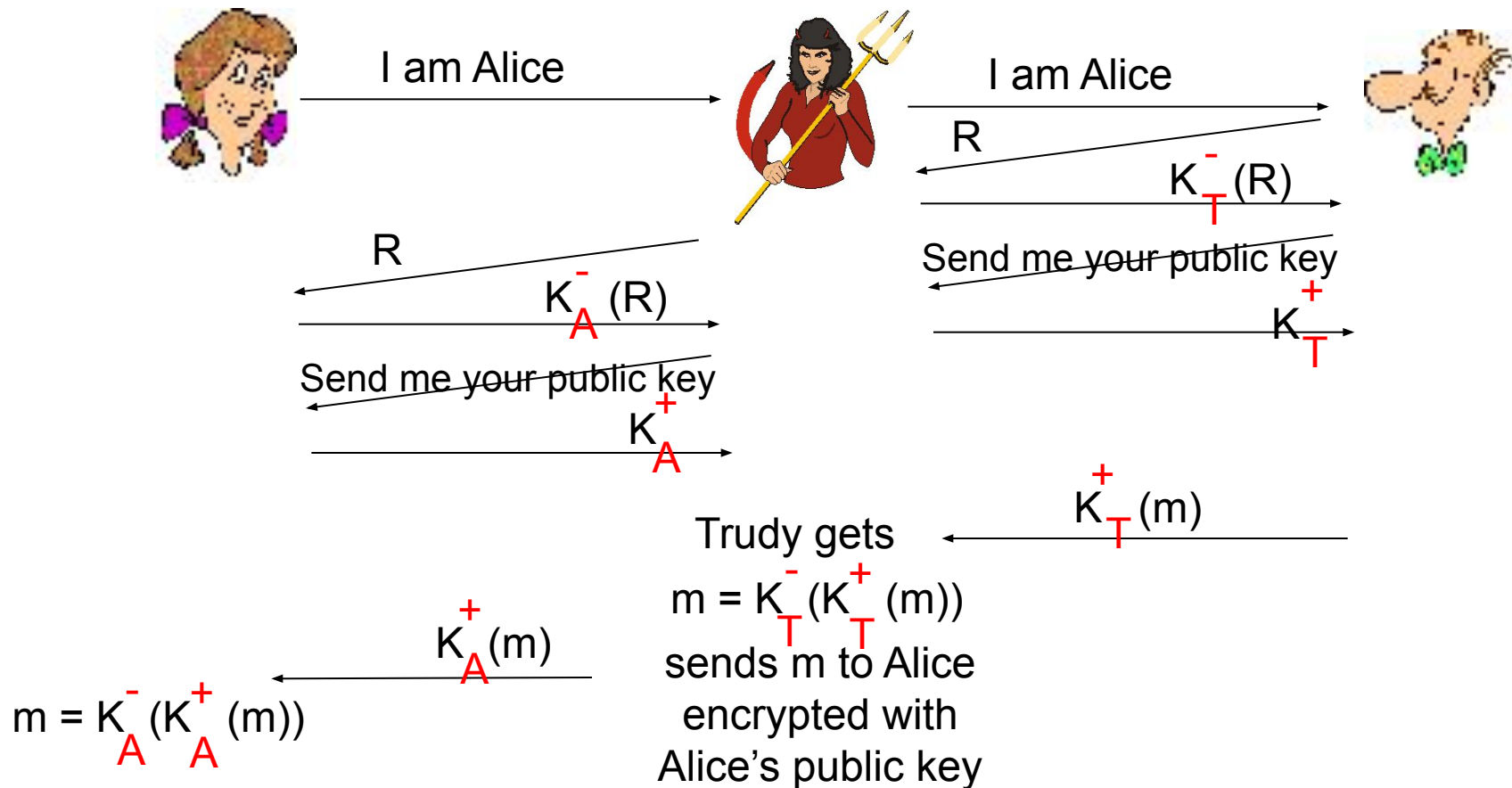


Hash function algorithms (Self Study)

- ❖ **MD5 hash function widely used (RFC 1321)**
 - computes 128-bit message digest in 4-step process.
 - arbitrary 128-bit string x , appears difficult to construct msg m whose MD5 hash is equal to x
- ❖ **SHA-1 is also used**
 - US standard [NIST, FIPS PUB 180-1]
 - 160-bit message digest

Recall: ap5.0 security hole

man (or woman) in the middle attack: Trudy poses as Alice (to Bob) and as Bob (to Alice)

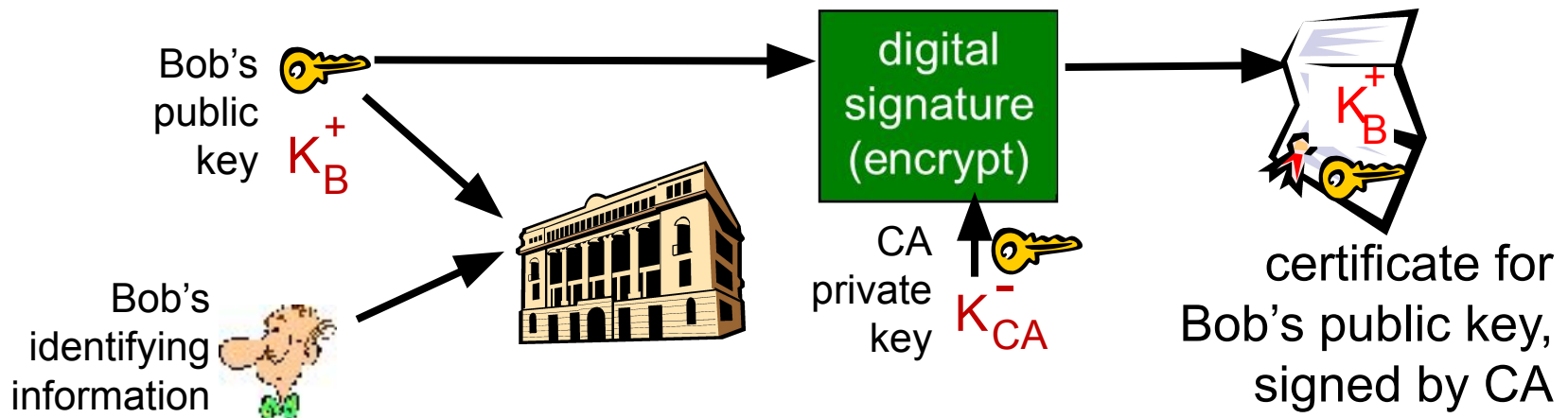


Public-key certification

- ❖ motivation: Trudy plays pizza prank on Bob
 - Trudy creates e-mail order:
Dear Pizza Store, Please deliver to me four pepperoni pizzas. Thank you, Bob
 - Trudy signs order with her private key
 - Trudy sends order to Pizza Store
 - Trudy sends to Pizza Store her public key, but says it's Bob's public key
 - Pizza Store verifies signature; then delivers four pepperoni pizzas to Bob
 - Bob doesn't even like pepperoni

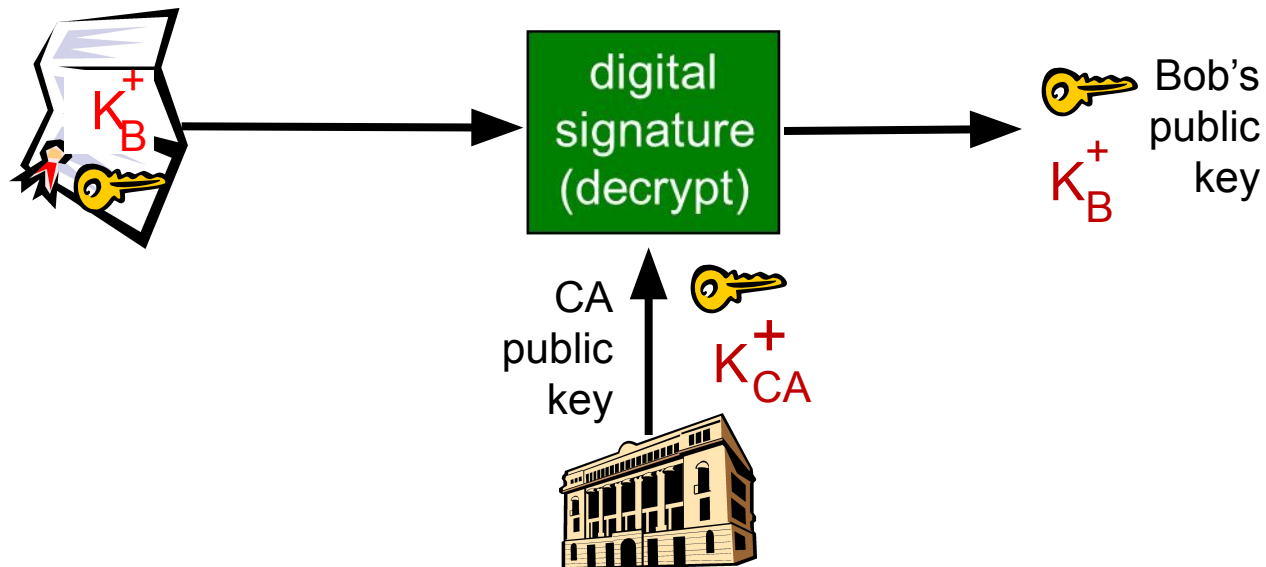
Certification authorities

- ❖ *certification authority (CA)*: binds public key to particular entity, E.
- ❖ E (person, router) registers its public key with CA.
 - E provides “proof of identity” to CA.
 - CA creates certificate binding E to its public key.
 - certificate containing E’s public key digitally signed by CA – CA says “this is E’s public key”



Certification authorities

- ❖ when Alice wants Bob's public key:
 - gets Bob's certificate (Bob or elsewhere).
 - apply CA's public key to Bob's certificate, get Bob's public key



Chapter 8 roadmap

8.1 What is network security?

8.2 Principles of cryptography

8.3 Message integrity, authentication

8.4 Securing e-mail

8.5 Securing TCP connections: SSL

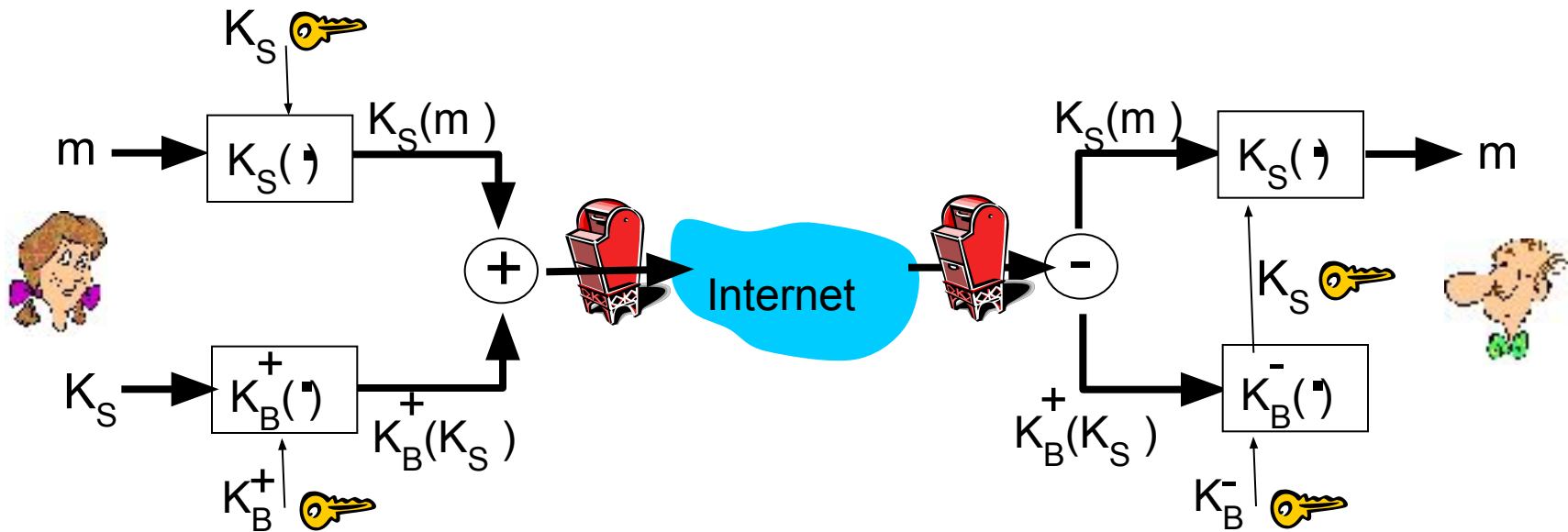
8.6 Network layer security: IPsec

8.7 Securing wireless LANs

8.8 Operational security: firewalls and IDS

Secure e-mail

- ❖ Alice wants to send confidential e-mail, m , to Bob.

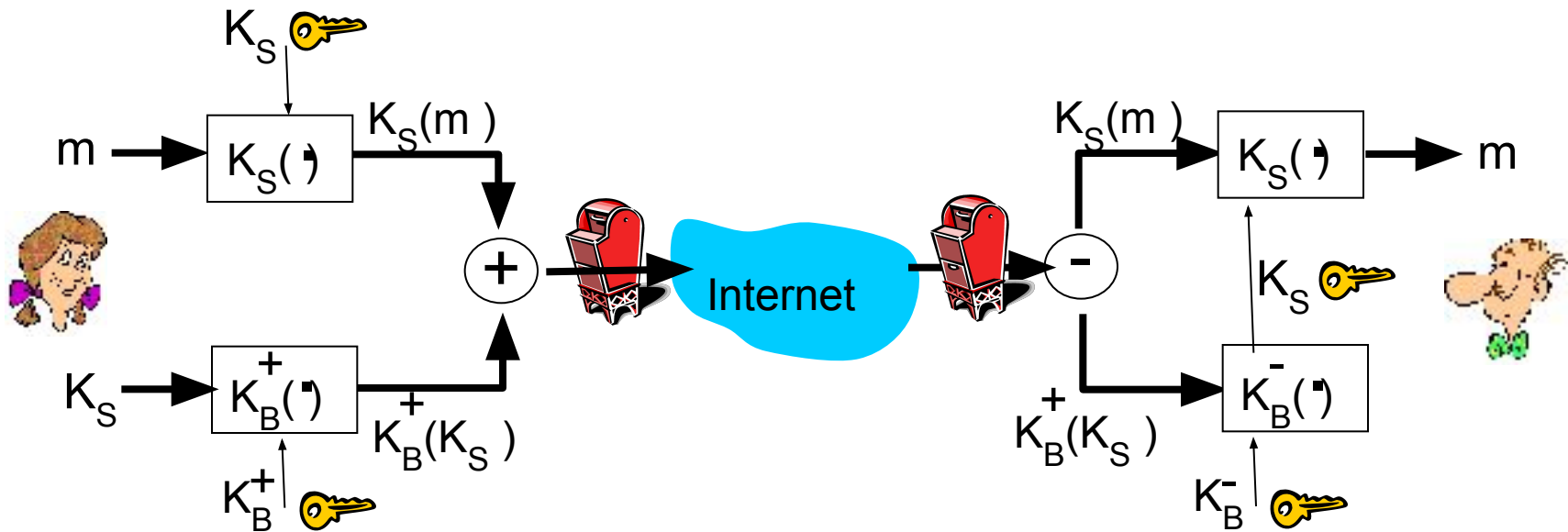


Alice:

- ❖ generates random *symmetric* private key, K_S
- ❖ encrypts message with K_S (for efficiency)
- ❖ also encrypts K_S with Bob's public key
- ❖ sends both $K_S(m)$ and $K_B(K_S)$ to Bob

Secure e-mail

- ❖ Alice wants to send confidential e-mail, m , to Bob.

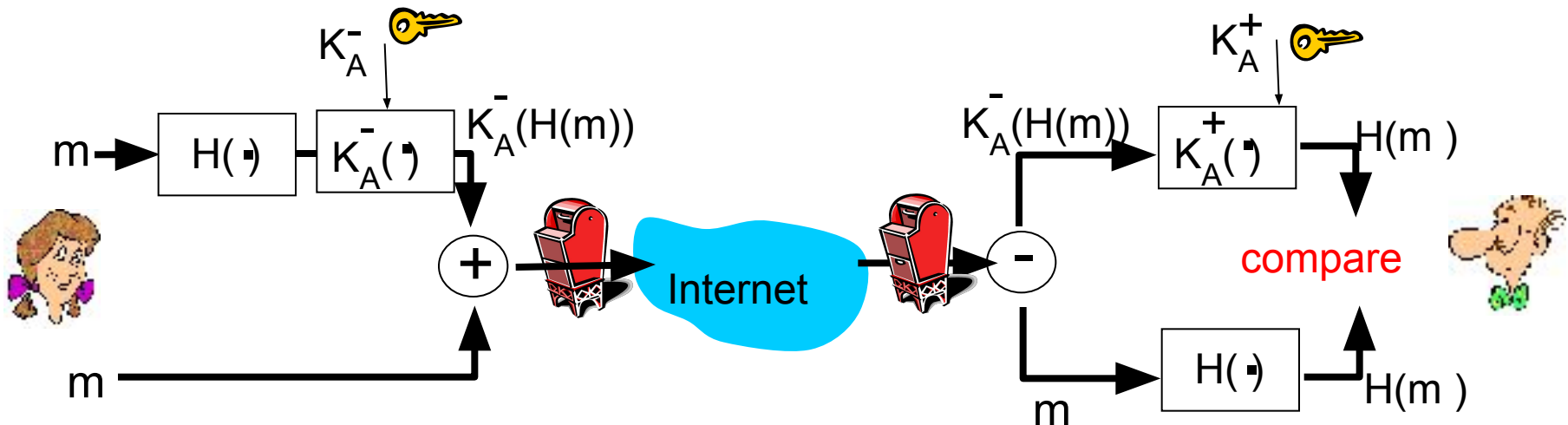


Bob:

- ❖ uses his private key to decrypt and recover K_S
- ❖ uses K_S to decrypt $K_S(m)$ to recover m

Secure e-mail (continued)

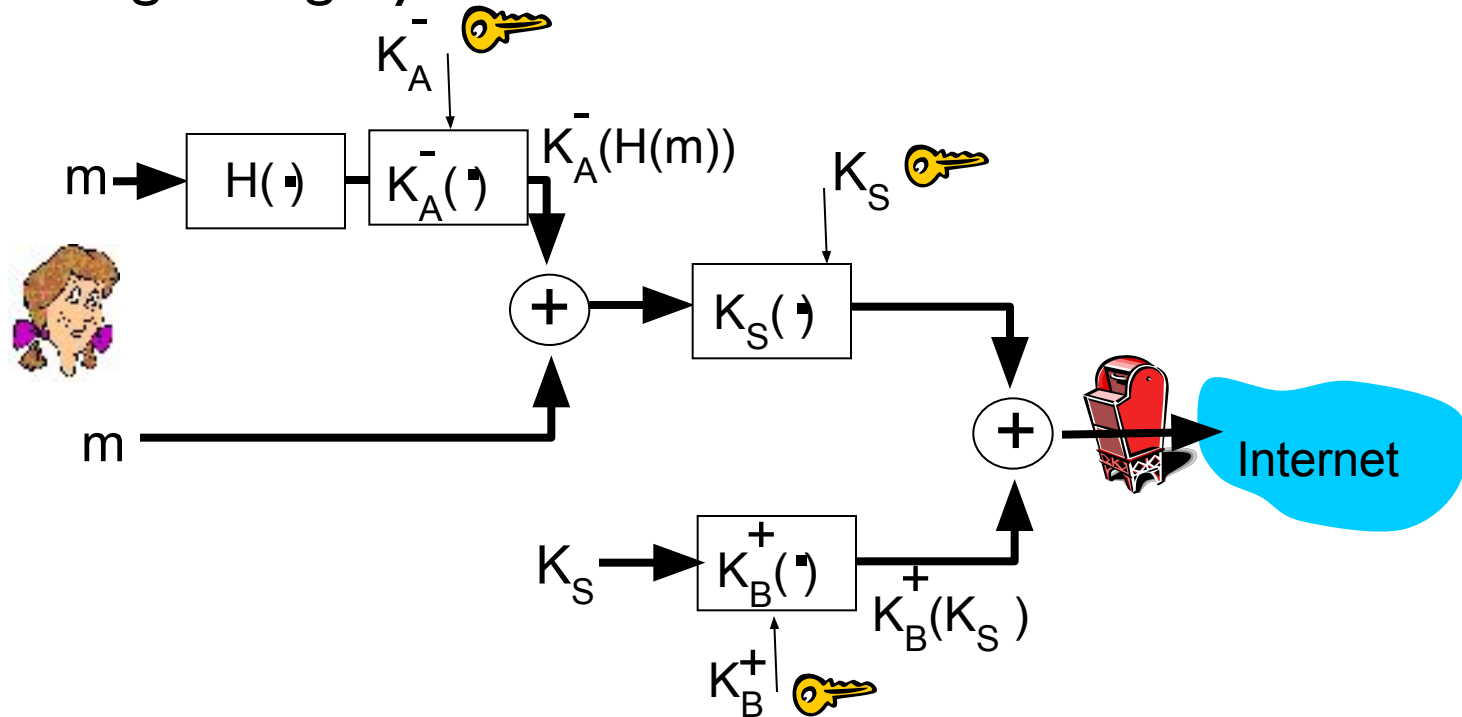
- ❖ Alice wants to provide sender authentication message integrity



- ❖ Alice digitally signs message
- ❖ sends both message (in the clear) and digital signature

Secure e-mail (continued)

- ❖ Alice wants to provide secrecy, sender authentication, message integrity.



Alice uses three keys: her private key, Bob's public key, newly created symmetric key